

1 Random and weighted sampling

Given a normalised probability density $P(x)$, we can write the average value, $\langle \mathcal{R} \rangle$ of some observable $\mathcal{R}$ as:

$$\langle \mathcal{R} \rangle = \int_L \mathcal{R}(x)P(x)dx \quad (1)$$

One way to (numerically) arrive at $\langle \mathcal{R} \rangle$ is to evaluate a weighted sum where x_i is sampled with a probability $w(x_i)$:

$$\langle \mathcal{R} \rangle = \frac{1}{N} \sum_{i=1}^N \mathcal{R}(x_i)P(x_i) \frac{1}{w(x_i)} \quad x_i \text{ drawn from } w(x)$$

The simplest way is to use a completely flat distribution, $w(x_i)$, so that x_i is sampled randomly within the region L :

$$\langle \mathcal{R} \rangle = \frac{L}{N} \sum_{i=1}^N \mathcal{R}(x_i)P(x_i) \quad x_i \text{ drawn randomly within } L$$

However, it is numerically more efficient to sample more often where $P(x)$ is large. Specifically, if we choose $w(x) = P(x)$, we have:

$$\langle \mathcal{R} \rangle = \frac{1}{N} \sum_{i=1}^N \mathcal{R}(x_i) \quad x_i \text{ drawn from } P(x)$$

An important example is Metropolis Monte Carlo sampling. Statistical thermodynamics have established that in a constant volume (V) and temperature (T), N -particle system, with co-ordinates $\mathbf{r}^N \equiv (x_1, y_1, z_1, x_2, \dots, x_N, y_N, z_N)$ and an interaction energy $U(\mathbf{r}^N)$, we obtain a Boltzmann-weighted probability distribution, i.e.:

$$P_B(\mathbf{r}^N) = \frac{e^{-\beta U(\mathbf{r}^N)}}{\int e^{-\beta U(\mathbf{r}^N)} d\mathbf{r}^N} \quad (2)$$

where β is the inverse thermal energy, i.e. $\beta = 1/(k_B T)$, where k_B is Boltzmann's constant. By drawing samples from this Boltzmann distribution we can establish (say) the average value for the interaction energy, $\langle U \rangle$ as:

$$\langle U \rangle = \frac{1}{M} \sum_{i=1}^M U(\mathbf{r}_i^N) \quad M \text{ samples, } (\mathbf{r}_i^N) \text{ drawn from } P_B$$

An obvious problem at this stage is that we do not know P_B , since the normalisation factor, i.e. the denominator, is unknown. The Metropolis procedure, which resolves this issue, will be described below.

2 Markov chains (briefly)

In a Markov chain, the transition probability from an "old" configuration, m , to a "new", n , is governed by a transition matrix, $\mathbf{T}$, with elements T_{nm} . This is perhaps most easily demonstrated by a classic example. Assume that the probability for a rainy day to be followed by a rainy day is

60%. On the other hand, the probability for a sunny day after a sunny day is 80%. This implies the following transition matrix:

$$\mathbf{T} = \begin{pmatrix} 0.6 & 0.2 \\ 0.4 & 0.8 \end{pmatrix} \quad (3)$$

Note that the column values add up to unity ($\mathbf{T}$ is a *stochastic* matrix). If we moreover know the state of day 1 to be rainy:

$$\boldsymbol{\rho}^{(1)} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad (4)$$

The weather prediction for day 2 thus becomes:

$$\boldsymbol{\rho}^{(2)} = \mathbf{T}\boldsymbol{\rho}^{(1)} = \begin{pmatrix} 0.6 \\ 0.4 \end{pmatrix} \quad (5)$$

as expected. Moreover,

$$\boldsymbol{\rho}^{(3)} = \mathbf{T}\boldsymbol{\rho}^{(2)} = \mathbf{T}^2\boldsymbol{\rho}^{(1)} = \begin{pmatrix} 0.44 \\ 0.56 \end{pmatrix} \quad (6)$$

The long-term weather prediction, $\boldsymbol{\rho}$, is independent of the initial state:

$$\boldsymbol{\rho} = \lim_{n \rightarrow \infty} \mathbf{T}^n \boldsymbol{\rho}^{(1)} = \begin{pmatrix} 1/3 \\ 2/3 \end{pmatrix} \quad (7)$$

This limiting distribution satisfies the eigenvalue equation:

$$\mathbf{T}\boldsymbol{\rho} = \boldsymbol{\rho} \quad (8)$$

i.e.

$$\sum_m T_{nm} \rho_m = \rho_n \quad (9)$$

This is often denoted a stationary, or *balance*, condition. A stronger, and useful, condition, is *detailed balance*: $T_{nm}\rho_m = T_{mn}\rho_n$. This automatically ensures the stationary condition:

$$\sum_m T_{nm} \rho_m = \sum_m T_{mn} \rho_n = \rho_n \sum_m T_{mn} = \rho_n \quad (10)$$

since $\mathbf{T}$ is a stochastic matrix.

3 The Metropolis Monte Carlo (MC) “simulations”

3.1 The basic scheme

This scheme was specifically developed to manage “simulations” of molecular systems, where we recall that the configurations are Boltzmann-distributed. Consider a trial move from m to n . Let α_{nm} represent the probability of attempting a $m \rightarrow n$ move. Let $P(m \rightarrow n)$ be the probability that an attempted $m \rightarrow n$ will be accepted. Then we can write our transition matrix elements as $T_{nm} = \alpha_{nm}P(m \rightarrow n)$. The criterion of detailed balance gives:

$$\alpha_{nm}P(m \rightarrow n)\rho_m = \alpha_{mn}P(n \rightarrow m)\rho_n \quad (11)$$

Guided by Metropolis *et al.* we choose $\boldsymbol{\alpha}$ to be symmetric ($\alpha_{nm} = \alpha_{mn}$). They chose to set:

$$P(m \rightarrow n) = \begin{cases} = \frac{\rho_n}{\rho_m}, & \rho_n < \rho_m \\ = 1, & \rho_n \geq \rho_m \end{cases} \quad (12)$$

Note that in a constant- (N, V, T) system, where states are P_B distributed (see eq. (2)), then $\rho_n/\rho_m = e^{-\beta(U_n - U_m)}$, i.e. $\rho_n < \rho_m$ implies that $U_n > U_m$, and vice versa. The transition matrix for elements $n \neq m$ are:

$$T_{nm} = \begin{cases} = \alpha_{nm} \frac{\rho_n}{\rho_m} & , \rho_n < \rho_m \\ = \alpha_{nm} & , \rho_n \geq \rho_m \end{cases} \quad (13)$$

completed by the possibility to remain in the same state:

$$T_{mm} = 1 - \sum_{n \neq m} T_{nm} \quad (14)$$

3.2 The MC algorithm in practice

We will at this stage assume an (NVT) system, i.e. a constant number of particles (N), volume (V) and temperature (T). A typical scheme can be summarised as:

1. Randomly select a particle m , with coordinate $\mathbf{r}_m$.
2. Make a trial displacement, to an attempted coordinate $\mathbf{r}_n$: $\mathbf{r}_n = \mathbf{r}_m + \Delta$, where Δ is a random vector, usually constrained within some limits (see below).
3. Calculate $\Delta U \equiv U_n(\mathbf{r}'^N; \mathbf{r}_n) - U_m(\mathbf{r}^N; \mathbf{r}_m)$.
4. Accept the move if $\xi < e^{-\beta \Delta U}$, where ξ is a random number ($0 < \xi < 1$), else retain the “old” coordinate. Note that a *new* random number has to be produced each time, i.e. ξ does not indicate a fixed number. If the move is accepted, update the coordinate for particle m , $\mathbf{r}_m \Rightarrow \mathbf{r}_n$, as well as the total energy (new energy: $U_n(\mathbf{r}'^N; \mathbf{r}_n) = U_m(\mathbf{r}^N; \mathbf{r}_m) + \Delta U$).
5. Occasionally, i.e. with some chosen (usually low) frequency, sample averages.
6. If the number of cycles is less than a chosen maximum, go back to entry 1; otherwise the simulation is finished, whereby averages are printed to some log file, and the final coordinates are saved on disk.

A common and simple choice for the attempted move is:

$$\begin{aligned} x_n &= x_m + \Delta(\xi_x - 0.5) \\ y_n &= y_m + \Delta(\xi_y - 0.5) \\ z_n &= z_m + \Delta(\xi_z - 0.5) \end{aligned} \quad (15)$$

where $\Delta \equiv |\Delta|$ (recall that a random number *never* should be re-used). How large should Δ be? In dense system, it is often chosen as less than a particle diameter, but in dilute systems it can be quite large. Note that a completely random attempted particle position $\mathbf{r}_n$ is a possibility, and in some systems an efficient sampling can be achieved by “mixing” regular attempted moves (as above) with occasional attempts to a random position. A common rule of thumb is to choose the attempted displacements so that the acceptance ratio becomes about 30 % or so.

3.3 Pairwise additivity

In order to calculate $U(\mathbf{r}^N)$, and thus the Boltzmann weights, $e^{-\beta U(\mathbf{r}^N)}$, interparticle interactions are in most cases assumed to be pairwise additive, i.e.

$$U(\mathbf{r}^N) = \sum_{i=1}^{N-1} \sum_{k=i+1}^N \Phi(\mathbf{r}_i, \mathbf{r}_k) \quad (16)$$

where Φ is the interparticle pair potential. Many-body interactions are usually neglected, or approximately taken into account by adjustments of Φ . An exception is polarization which can be accounted for, albeit at a considerable computational expense.

3.4 Periodic boundary conditions

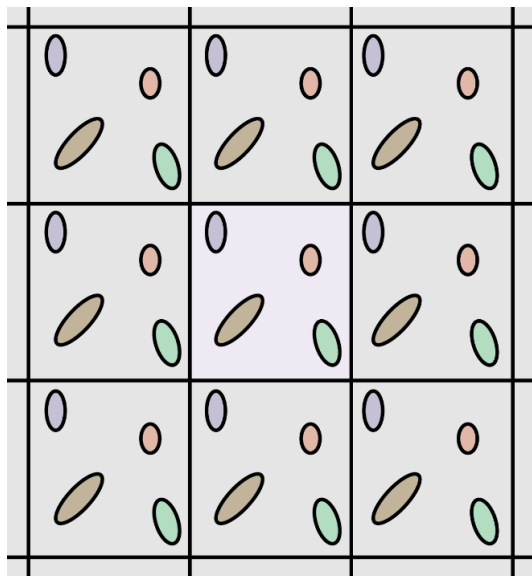


Figure 1: Periodic boundary conditions, in 2D. Copied from Wikimedia Creative Commons: https://commons.wikimedia.org/wiki/File:Periodic_Boundary_Conditions_in_2D.png. Original graphics by Christopher Rowley, CC BY-SA 4.0 <<https://creativecommons.org/licenses/by-sa/4.0/>>, via Wikimedia Commons

The past few decades have seen a dramatic increase of the computational capacity of modern computers. Nevertheless, it is still a time consuming task to simulate classical systems containing a few thousand molecules. Typical macroscopic systems of interest contains about 10^{24} molecules. How are we able to bridge this enormous difference? First of all, we are aided by Statistical Thermodynamics which tells us that only *average* quantities are of interest. Temperature, for instance, is related to the average speed of the molecules, but the speed of a particular molecule at a given instant is irrelevant. Fortunately, averages tend to converge rapidly, and we can

actually obtain relevant information from simulations of very small systems. However, if we want to sample bulk properties, we have to avoid *surface effects*: for a small system the influence of boundary surfaces is unrealistically large. We can tackle this with a trick called *periodic boundary conditions*: the cubic (say) simulation box is replicated to form an infinite lattice of such boxes. This is illustrated in Figure 1. As a particle moves in the (central) simulation box, all replica particles are also moved. If the “central” particle moves out of the central simulation box (attempted coordinates, eq. (15)), it is replaced by an identical one at the opposing position, within the simulation box. In this way, boundary surfaces are effectively removed (at the expense of imposing an artificial periodicity).

Note that when we attempt to move a particle i , then the simulation box is effectively centred on i , and i thus interacts with the closest periodic replica of another particle k . In other words, if you find that (say) $x_{ik} = -0.63L$ (note: $|x_{ik}| > L/2$) then you shift this x separation to $x_{ik} = -0.63L + L = 0.37L$ which represents the nearest $|\Delta x|$ distance in the periodically replicated system.

3.5 Model potentials

Unfortunately proper evaluations of intermolecular potentials require quantum mechanical approaches, which are computationally very demanding. Hence, in order to facilitate classical simulations, i.e. reasonable system sizes, it is very common to use simple classical model potentials. For instance, all atoms interact via an attractive *dispersion interaction* at long range, and a repulsive *exchange repulsion* at short range. These, though of quantum mechanical origin, are often approximated by a “Lennard-Jones” (L-J) potential:

$$\Phi(r) = 4\epsilon\left[\left(\frac{\sigma}{r}\right)^{12} - \left(\frac{\sigma}{r}\right)^6\right] \quad (17)$$

where r is the separation, σ reflects the “size” of the particles (possibly atoms, but the description could be more coarse-grained), while $-\epsilon\sigma^6$ determines the strength of the attractive interaction (σ^3 can be an approximation of the electronic polarisability). The minimum value of the interaction potential is $-\epsilon$. The potential is illustrated in Figure 2.

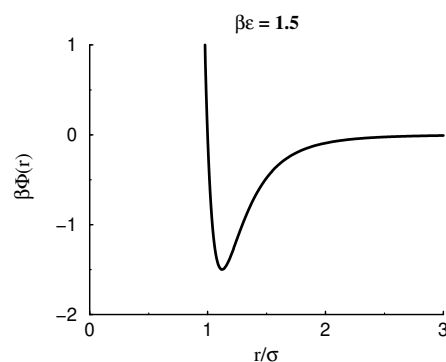


Figure 2: The Lennard-Jones potential.

This is a very crude description, but it does have the correct algebraic decay at long range (disregarding relativity effects), and also displays a steep repulsion as the atoms are brought very

close together. Similarly, the polar nature of water is often approximated via distributed partial point charges.

3.6 Comparing with Molecular Dynamics simulations

The MC technique has some clear advantages: it is easy to swap between ensembles (changing macroscopic constraints, such as constant pressure instead of constant volume) and there are clever ways to avoid being stuck in local free energy minima, where one option is occasional “unphysically large” attempted moves. Nevertheless, it does have one significant drawback: since time never enters, we are unable to calculate dynamical quantities, such as diffusion ¹. Fortunately, there is an alternative, and in many ways more straightforward, method to calculate thermodynamic averages. In this case, we actually “simulate” the physical system, i.e. follows the time evolution. Here, thermodynamic quantities (such as the pressure) is calculated as time averages. The method is called *Molecular Dynamics*, MD, and it is directly based on Newton’s equation of motion:

$$m_i \frac{d^2 \mathbf{r}_i}{dt^2} = - \sum_{i \neq j, j=1}^N \mathbf{F}_{ij} \quad (18)$$

where $\mathbf{F}_{ij}$ is the force acting between molecules (or atoms) i and j .

Eq. (18) is solved numerically, via Taylor expansions, whereby the system moves step-wise between times t and $t + \delta t$. Note that this method is potentially unstable, i.e. we have to make sure that the error associated with a discrete time step is small enough to make a negligible contributions to the averages we seek. In other words, the time step has to be small, particularly in dense systems. This makes it computationally very demanding to simulate, say, protein dynamics in a model system with explicit water molecules. This is because our time step is strongly limited by the fast water dynamics, but we still have to cover a time scale relevant for protein dynamics. MD comprises what one might call a “proper” simulation. MC, on the other hand, is arguably best described as a very efficient way to calculate multidimensional integrals.

We can summarise the pros and cons of MD, compared with MC, as follows:

- (+) allows calculations of dynamical quantities, and non-equilibrium processes
- (+) often lends itself to efficient parallelisation, since each time step amounts to N^2 calculations (movement of N particles) as compared with N calculations for a typical MC implementation of an attempted move
- (-) the stability issue mentioned above needs to be considered
- (-) “unphysical” moves that are permitted in MC, are not allowed here, and this may lead to entrapment in local free energy minima
- (-) the method requires calculations of forces, which can be an issue for some models, such as hard spheres

Broadly speaking, MD is usually preferred for detailed models, including those where each atom is explicitly modelled in some way (“all-atomistic”). MC is often the method of choice when more coarse-grained models are utilised, unless dynamic properties are of interest.

¹There are actually “constrained” MC versions, with imported dynamical information, that can be used to estimate dynamics, but these will not be considered here.

3.7 Ensembles and macroscopic constraints

This is not a course in Statistical Mechanics, but it is nevertheless appropriate to briefly mention an important connection between static and dynamic systems. An *ensemble* is a very large collection of systems, that differ at the microscopic level, but are defined by the same macroscopic constraints. The ergodic hypothesis states that the instantaneous average of a *mechanical* variable across the ensemble of systems, is identical to the long time average within one member of the ensemble. A *mechanical* variable is defined for a single configuration. Examples include pressure (p), volume (V), particle number (N), and potential energy (U). Examples on *non-mechanical* variables: temperature, entropy, and free energies. Different macroscopic constraints lead to different ensembles. For instance, our constant (N, V, T) constraint defines the *canonical* ensemble. With constant (N, p, T) we arrive at the *isobaric* ensemble. We shall use the isobaric ensemble to illustrate that the MC algorithm differs between ensembles. Another motivation is that this ensemble is utilised in one of the suggested simulation projects. We simplify things by assuming that we simulate a bulk system, using a cubic box with a volume $V = L^3$. Note that, in order to adhere to the constraint of a constant pressure, the volume is a fluctuating quantity. The configurational average of the observable $\mathcal{R}$ can in the isobaric ensemble be expressed as:

$$\langle \mathcal{R} \rangle = \frac{\int_0^\infty \int \mathcal{R} e^{-\beta(U(\mathbf{r}^N) + pV)} d\mathbf{r}^N dV}{\int_0^\infty \int e^{-\beta(U(\mathbf{r}^N) + pV)} d\mathbf{r}^N dV} = \frac{\int_0^\infty V^N e^{-\beta pV} \int \mathcal{R} e^{-\beta(U(\mathbf{s}^N; L))} d\mathbf{s}^N dV}{\int_0^\infty \int e^{-\beta(U(\mathbf{r}^N) + pV)} d\mathbf{r}^N dV} \quad (19)$$

where we have introduced scaled coordinates, $\mathbf{s}$, where

$$\mathbf{s} = \frac{\mathbf{r}}{L} \quad (20)$$

The notation $U(\mathbf{s}^N; L)$ indicates that the interaction energy depends on real, rather than scaled, coordinates. In this ensemble, standard attempted particle displacements, as discussed above, must be complemented by attempts to change the volume of the system. Here, an attempted new volume V_n is typically generated as:

$$V_n = V_m + \Delta_V (\xi - 0.5) \quad (21)$$

where V_m is the existing (“old”) volume, and the parameter Δ_V often is adjusted so that an acceptance ratio for volume fluctuations is about 30 %. The limiting distribution of the Markov chain is proportional to $V^N e^{-\beta pV} e^{-\beta(U(\mathbf{s}^N; L))} = e^{(-\beta(U(\mathbf{s}^N; L) + pV) + N \ln[V])}$. Thus, we now accept the move if

$$\xi < e^{(N \ln[V_n/V_m] - \beta(U_n - U_m + p(V_n - V_m)))} \quad (22)$$

Note that all real (unscaled) coordinates change, i.e. $\mathbf{r}_m = \mathbf{s}L_m \rightarrow \mathbf{r}_n = \mathbf{s}L_n$. This normally implies $N(N-1)/2$ interparticle potential energy calculations, i.e. such attempted volume fluctuations are computationally expensive, and should be performed with a rather low frequency. However, in the special case where the interaction potential is spherically symmetric, and non-truncated, a volume fluctuation is extremely cheap. This is because the interparticle distance, $r_{ij} \equiv |\mathbf{r}_j - \mathbf{r}_i|$, between (say) i and j , changes from $r_{ij}^{(m)} = L_m s_{ij}$ to $r_{ij}^{(n)} = L_n s_{ij}$. Thus, $\frac{r_{ij}^{(m)}}{r_{ij}^{(n)}} = \frac{L_m}{L_n}$. In other words, if we simulate a Lennard-Jones system, and keep track of the repulsive ($U^{(12)}$) and attractive ($U^{(6)}$) energy terms separately, then the change of the total Lennard-Jones energy (U) upon an attempted volume fluctuation becomes

$$U_n - U_m = U_m^{(12)} \left(\frac{L_m}{L_n}\right)^{12} + U_m^{(6)} \left(\frac{L_m}{L_n}\right)^6 - U_m \quad (23)$$

3.8 Equilibration and sampling

How do we start a simulation? One option is to assign lattice-based initial coordinates. Another is to insert all particles at random, within the simulation box. In both cases, the initial state is probably a poor representation of a typical equilibrium configuration. Hence, we postpone any sampling of equilibrium quantities until the system has converged enough to provide typical equilibrium states. A common way to monitor the approach to equilibrium is to plot “snapshots” of the total energy, U . When a plateau value is established, one can assume that the system has at least reached a metastable state (and in most cases equilibrium). An example is given in

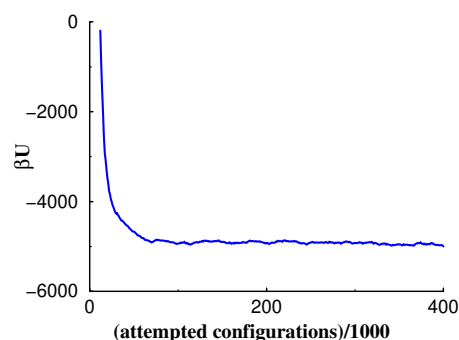


Figure 3: Sampling snapshot configurational energies (U) of a system, as it approaches equilibrium (the plateau region). Initial positive values of U are excluded. The system was initialised with random coordinates.

Figure 3. In this case, we should only sample averages after about 10^5 attempted configurations. It is recommended to write the final coordinates from a simulation to disk, since they hopefully represent typical equilibrium coordinates. These can then be used as (properly equilibrated) starting coordinates, if you choose to continue the simulation.

3.9 Energy conservation

A very useful and sensitive way to verify that your MC code produces correct results, is to make occasional checks of energy conservation. You may, for instance, divide your simulation into (say) ten equal and sequential parts, each comprising 10 % of the total number of attempted moves. After each “sub-simulation” you recalculate the total interaction energy and also make other suitable checks, such as coordinates remaining inside the simulation box, no hard-sphere overlaps (if the model system contains hard spheres) etc. The recalculated total interaction energy should of course match the one you ended up with after the sub-simulation. An acceptable relative discrepancy is $|\delta U/U| \approx 10^{-11}$ or less (assuming a double precision floating point format).